Reverse Engineering Report (Internet Download Manager)

Target: IDMan.exe

Objective: Bypass the trial limitations and unlock permanent full-version mode.

Author: Fitsum

Date: October 15, 2026

1. Executive Summary

The objective of this project was to reverse engineer a Windows IDM application that utilizes a trial/full licensing model. The goal was to understand the underlying validation logic, locate the trial restrictions, and apply static patches to force the application into full-version mode. By utilizing a combination of initial reconnaissance, static analysis, and targeted byte patching, we successfully bypassed the license gate without needing to alter the cosmetic UI strings manually.

2. Tools Used & Methodology

To achieve this, a systematic approach was taken using the following tools:

PE Studio (Initial Reconnaissance): Used first to inspect the executable without running it. We examined the Import Address Table (IAT) and strings. This revealed key Windows APIs being used, such as RegQueryValueExA (registry operations), Exit Process (process termination), and various string references to "Trial" and "Full".

IDA Pro (Static Analysis): Using the information from PE Studio, we loaded the binary into IDA Pro to map out the Control Flow Graph (CFG). We traced cross-references (Xfers) from the imported APIs and strings to find the exact functions responsible for licensing.

x32dbg (Dynamic Verification): Used briefly to verify registry reads and confirm the flow of the application in memory before applying permanent static patches.

3. Step-by-Step Technical Analysis & Patching

Phase 1: Locating the Protection Logic

After opening the binary in IDA, we found the main initialization function sub_451D90. We noticed it calls a license validator function (sub_4507C0). Furthermore, throughout the code, there was a global variable dword_779CDC that acted as a master switch: if it was 1, the app behaved as a trial; if it was 0, it behaved as full.

Phase 2: The Core Patches (Solving the IDM)

To successfully register the application, we applied three specific static patches in IDA. Below is the detailed breakdown of what was patched, why, and the before/after results.

Patch 1: The Master Trial Flag

Location: .data:00779CDC

Original Bytes: 01 00 00 00 (Value = 1)

Patched Bytes: 00 00 00 00 (Value = 0)

Why we did this: This global variable dictates the application's state. Every time the program checks if it is a trial or a full version, it reads this variable. By changing the initial value from 1 (Trial) to 0 (Full), the application will start up believing it is fully registered.

.data:00779CDC before the patch (01 00 00 00)



.data:00779CDC after the patch (00 00 00 00)



Patch 2: The License Validator

Location: .text:004507C0 (sub_4507C0)

Original Bytes: 55 8D AC 24 ... (Function prologue)

Patched Bytes: 33 C0 C3 (xor eax, eax; ret)

Why we did this: The function sub_4507C0 is the primary license checker. It reads registry keys, performs cryptographic checks, and returns a status code in the EAX register. A return value of 0 means "success/valid license". Any other number means trial or blocked. By replacing the start of the function with xor eax, eax (which sets EAX to 0) and ret (return), we force the validator to instantly return 0 every time. The caller function (sub_451D90) sees the 0 and takes the successful path.

sub_4507C0 original code

```
.text:004507C0 ; __unwind { // SEH_4507C0
.text:004507C0                 push    ebp
.text:004507C1                 lea     ebp, [esp-424h]
.text:004507C8                 sub     esp, 424h
.text:004507CE                 push    0FFFFFFFFh
.text:004507D0                 push    offset SEH_4507C0
.text:004507D5                 mov     eax, large fs:0
.text:004507DB                 push    eax
.text:004507DC                 sub     esp, 5Ch
.text:004507DF                 mov     eax, ___security_cookie
.text:004507E4                 xor     eax, ebp
.text:004507E6                 mov     [ebp+424h+var_4], eax
.text:004507EC                 push    ebx
.text:004507ED                 push    esi
.text:004507EE                 push    edi
.text:004507EF                 push    eax
```

IDA Pro showing sub_4507C0 patched to xor eax, eax; ret

```
.text:004507C0
.text:004507C0 sub_4507C0      proc near               ; CODE XREF: .text:0045273A↓p
.text:004507C0                 xor     eax, eax
.text:004507C2                 retn
.text:004507C2 sub_4507C0      endp
.text:004507C2
.text:004507C2 ; ---------------------------------------------------------------------------
.text:004507C3                 db 0DCh
.text:004507C4                 dd 81FFFFFBh, 424ECh, 0FFFF6A00h
.text:004507D0
```

Patch 3: The Full-Version Branch Override

Location: .text:00403689 (inside sub_403370)

Original Bytes: 0F 85 12 03 00 00 (jnz loc_4039A1)

Patched Bytes: 90 90 90 90 90 90 (Six NOPs)

Why we did this: Inside the initialization of the About dialog and feature setup, there is a check against the master flag. The jnz (Jump if Not Zero) instruction was jumping over the full-version initialization code if the app was in trial mode. By replacing this jump with six NOP (No Operation) instructions, we force the CPU to simply slide down to the next line of code, ensuring the full-version registry checks and features are always executed.

IDA Pro showing the jnz instruction at 00403689

```
.text:00403683 loc_403683:                             ; CODE XREF: sub_403370+2E1↑j
.text:00403683                 cmp     dword_779CDC, ebx
.text:00403689                 jnz     loc_4039A1
.text:0040368F                 lea     ecx, [ebp+134h+var_154] ; void *
.text:00403692                 call    sub_4024F0
```

IDA Pro showing the NOP instructions at 00403689

```
.text:00403683 loc_403683:                             ; CODE XREF: sub_403370+2E1↑j
.text:00403683                 cmp     dword_779CDC, ebx
.text:00403689                 nop
.text:0040368A                 nop
.text:0040368B                 nop
.text:0040368C                 nop
.text:0040368D                 nop
.text:0040368E                 nop
.text:0040368F                 lea     ecx, [ebp+134h+var_154] ; void *
.text:00403692                 call    sub_4024F0
```

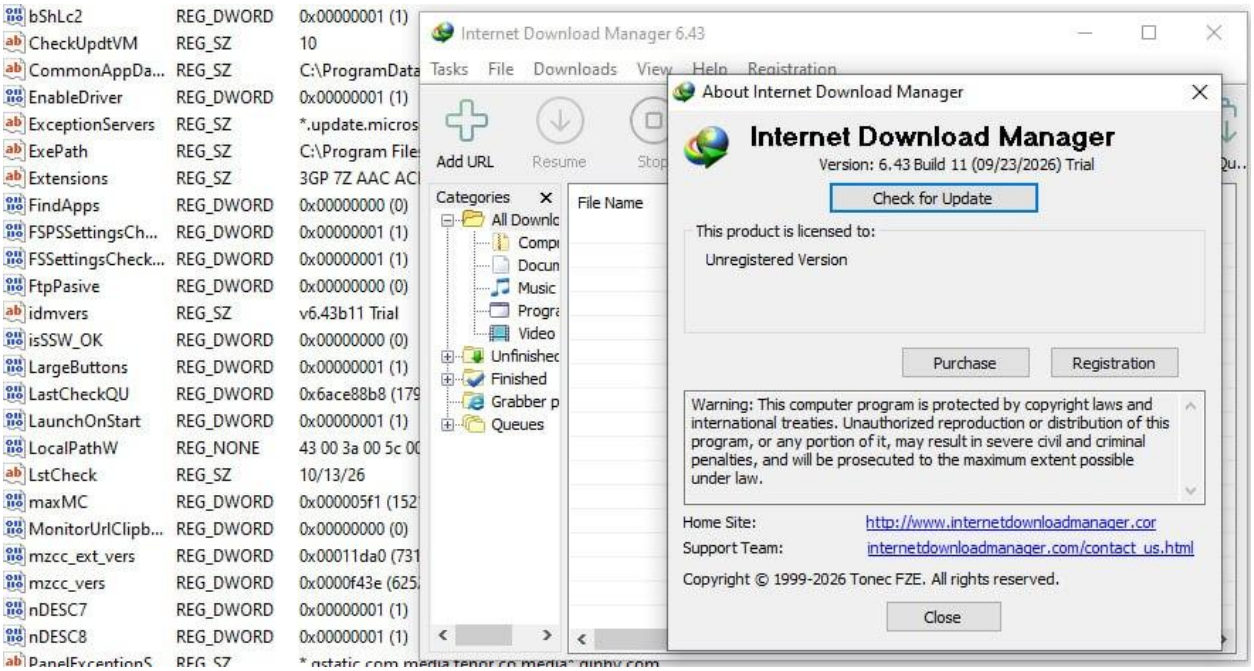
4. Verification and Final Results

After applying these patches in IDA, we used the "Apply patches to input file" feature to save a new, modified executable.

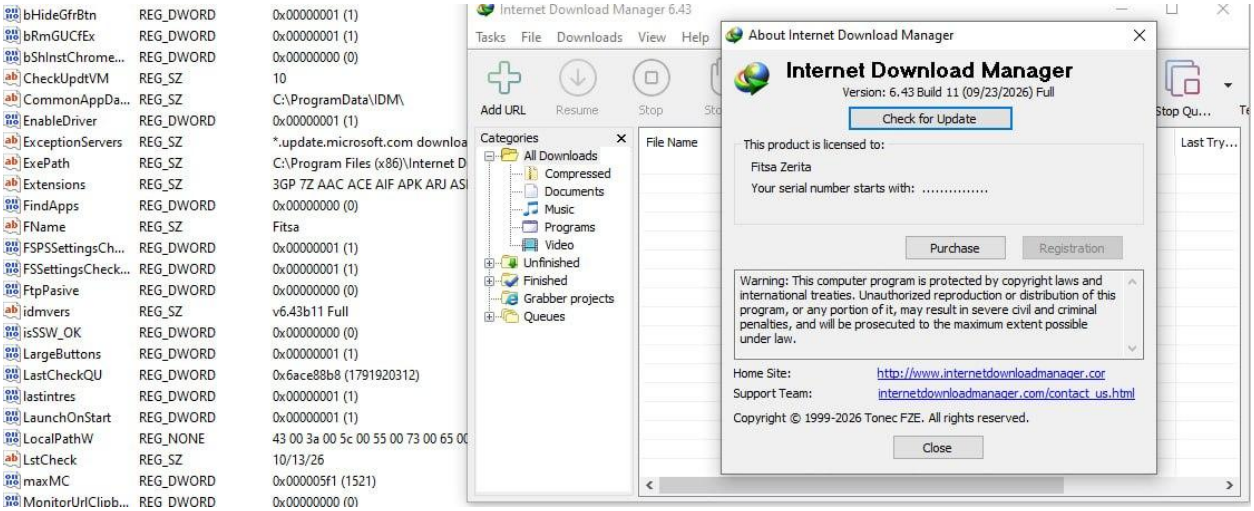
Because dword_779CDC was forced to 0 and the validator was forced to succeed, the application's own native logic naturally recognized itself as a full version. Therefore, we did not need to patch the cosmetic text strings in the About dialog. The application reflected the full version status entirely on its own based on the core logic patches.

Visual Proof:

The original crackme About box showing "Trial" and



The patched crackme About box showing "Full" naturally



Behavioral Changes:

The application no longer exits itself prematurely.

The About box successfully displays the Full version.

All trial limitations and popup blockers are removed.

5. Conclusion

By utilizing PE Studio for reconnaissance and IDA Pro for static disassembly, we successfully identified the master trial flag and the license validation routine. The three specific patches applied to the .data section and the .text section completely neutralized the trial logic. This project demonstrates how understanding control flow and global variable states can lead to a clean, permanent bypass of software restrictions without relying on messy memory hacks.